MODULE 11

Agents for Software Development: Applied

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 2 / Third**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Use Claude Code for systematic repository observation and codebase understanding before making changes
2. Apply intelligent file localization strategies using Grep, Glob, and @-references to navigate large codebases efficiently
3. Execute the full Plan-Execute-Review cycle using Claude Code's plan mode with human approval gates
4. Define and measure safety and evaluation metrics for agent-generated code including diff size, test pass rate, and coverage delta
5. Configure MCP (Model Context Protocol) servers to extend Claude Code's capabilities beyond the local filesystem
6. Combine subagents, plan mode, and MCP tools into a complete applied software development workflow

SESSION TIMING BREAKDOWN

0:00 – 0:08	**Section 1: Repository Observation & Codebase Understanding**
0:08 – 0:16	**Section 2: File Localization — Intelligent Grep & Glob**
0:16 – 0:26	**Section 3: The Plan-Execute-Review Cycle**
0:26 – 0:34	**Section 4: Safety & Evaluation Metrics**
0:34 – 0:43	**Section 5: MCP Server Integration**
0:43 – 0:45	**Wrap-Up: Quick Reference & Transition to Lab 2**

0:00 – 0:08	SECTION 1: Repository Observation & Codebase Understanding

1. Repository Observation & Codebase Understanding

1.1 Observe Before You Act

The #1 mistake in agent-assisted development is jumping straight to implementation. Before any agent writes code, it must understand the codebase. Claude Code's observation phase reads project structure, CLAUDE.md, existing patterns, and test conventions to make informed decisions.

```
# Step 1: Ask Claude to observe the project
> Analyze this repository. Tell me:
> 1. Project structure and key directories
> 2. Tech stack and dependencies (pyproject.toml)
> 3. Coding patterns used in existing modules
> 4. Test organization and fixture patterns
> 5. Any CLAUDE.md rules I should know about

# Step 2: Ask about a specific area before changing it
> Before I modify validators.py, analyze:
> - Which other modules import from it?
> - What tests cover it and what's the current coverage?
> - Are there any edge cases in the existing implementation?
```

Without Observation	With Observation
"Add a humidity validator" → Claude creates a new function that duplicates existing logic	"First analyze validators.py, then add humidity validation following the same pattern" → consistent code
Agent writes code in wrong directory	Agent reads project structure first, places files correctly
Agent ignores existing test patterns	Agent reads conftest.py first, uses existing fixtures
Generated code conflicts with CLAUDE.md rules	Agent reads CLAUDE.md first, follows team standards

☆ **KEY POINT:** Train your agents to observe first. In your subagent system prompts (Module 10), always start with: "When invoked: 1. Read the project's CLAUDE.md. 2. Analyze the target file and its imports. 3. Review existing test patterns. THEN proceed with the task."

0:08 – 0:16	SECTION 2: File Localization — Intelligent Grep & Glob

2. File Localization: Intelligent Grep & Glob

2.1 Finding the Right Files

In large codebases, knowing which files to read and modify is half the battle. Claude Code has built-in tools for file localization, and you can guide it with smart strategies:

Strategy	Claude Code Tool	Example
Find by pattern	Glob	Glob("src/**/*validator*.py") — all validator files
Find by content	Grep	Grep("SensorReading", "src/") — files using the dataclass
Find by reference	@-reference	@src/sensor_toolkit/validators.py — direct file read
Find by import	Grep	Grep("from sensor_toolkit.validators import", ".") — all importers
Find by test	Glob + Grep	Glob("tests/**/test_*.py") then Grep for function name
Find by change	Bash	git log --all --oneline -- src/validators.py — change history

2.2 Localization Prompts

```
# Find all files affected by a change:
> Before modifying the SensorReading dataclass, find:
> 1. Every file that imports SensorReading
> 2. Every test that creates a SensorReading instance
> 3. Every function that takes SensorReading as a parameter
> List file paths and line numbers.

# Navigate by architecture layer:
> Show me the data flow: which file reads raw CSV,
> which validates, which cleans, which analyzes?
> Draw the dependency graph as a list.

# Find related tests for a function:
> Find all tests that directly or indirectly test
> the validate_reading() function. Include tests that
> test validate_batch() since it calls validate_reading().
```

☆ **KEY POINT:** When you add localization to your subagent prompts, agents make targeted changes instead of broad rewrites. A planner subagent that first runs Grep("SensorReading") knows exactly which files are affected before proposing changes. This reduces diff size and prevents unintended side effects.

Track A (IT Engineers)	Track B (Process Engineers)
Localize API endpoints: Grep for @app.route, trace request handlers to DB queries	Localize data pipeline: trace CSV input → validation → cleaning → output. Map file dependencies.

Find all error handlers: Grep for except and raise across the codebase. Audit error handling patterns.	Find all hardcoded thresholds: Grep for numeric literals in validators.py and cleaners.py.

0:16 – 0:26	SECTION 3: The Plan-Execute-Review Cycle

3. The Plan-Execute-Review Cycle

3.1 Plan Mode

Plan mode (Shift+Tab) tells Claude to think through a change without executing it. This is the foundation of safe agent-assisted development: plan first, human approves, then execute.

Phase	What Happens
1. Plan	Claude analyzes the task, reads relevant files, and proposes a detailed plan: which files to change, what changes to make, and why.
2. Human Review	You review the plan. Approve, modify, or reject. This is the approval gate.
3. Execute	Claude implements exactly the approved plan. No surprise changes.
4. Test	Run tests immediately after execution. Verify nothing broke.
5. Review	Check the diff (git diff). Verify changes match the plan.
6. Commit or Rewind	If tests pass and diff looks good: commit. Otherwise: /rewind.

```
# Activate plan mode:
> Shift+Tab  (toggles plan mode on/off)

# In plan mode, ask for a change:
> [PLAN MODE] Add a humidity_offset parameter to
> the clamp_outliers function that allows calibration
> adjustment for different sensor models.

# Claude responds with a plan:
# - Modify clamp_outliers() signature in cleaners.py
# - Add humidity_offset parameter with default=0
# - Adjust clamping logic to include offset
# - Update 2 tests in test_cleaners.py
# - Update docstring

# Review and approve:
> Looks good. Execute this plan.

# Claude implements exactly the plan
> !pytest tests/test_cleaners.py -v  # Verify
> !git diff  # Check changes match plan
```

☆ **KEY POINT:** Plan mode is the most important safety mechanism for agent-assisted development. It creates an explicit approval gate between “what should we do” and “do it.” In production workflows, always use plan mode for non-trivial changes. Quick fixes (typos, import fixes) can skip it.

3.2 The Full Cycle with Subagents

```
# Complete Plan-Execute-Review with agents:
```

```
# 1. Planner subagent observes and plans (read-only)
> Use the sensor-planner subagent to create a plan for
> adding humidity calibration offset support.

# 2. Review the plan output
> Show me the plan the planner created.

# 3. Implementer subagent executes (with write access)
> Use the sensor-implementer subagent to execute the
> plan in docs/plan.md.

# 4. Tester subagent verifies (runs tests)
> Use the sensor-tester subagent to verify the changes.

# 5. Reviewer subagent audits (read-only)
> Use the sensor-reviewer subagent to review the diff.
```

0:26 – 0:34	SECTION 4: Safety & Evaluation Metrics

4. Safety & Evaluation Metrics for Agent-Generated Code

4.1 Measuring Agent Output Quality

How do you know if an agent's output is good? You need measurable metrics that can be checked automatically:

Metric	How to Measure	Acceptable Threshold
Diff size	git diff --stat | tail -1	< 200 lines for a single feature
Test pass rate	pytest --tb=short | grep "passed"	100% (all existing tests pass)
Coverage delta	pytest --cov before vs. after	Coverage stays same or increases
Lint violations	ruff check src/ --output-format json | jq '.length'	Zero new violations
Type check	mypy src/ --strict | grep "error"	Zero new type errors
Complexity	radon cc src/ -a -nc	No new functions above C complexity

4.2 Automated Quality Gates

```
# Quality gate script for agent output:
#!/bin/bash
echo '=== Agent Output Quality Gate ==='

# 1. Diff size check
LINES=$(git diff --stat | tail -1 | grep -oP '\d+ insertion'| grep -oP '\d+')
[ "$LINES" -gt 200 ] && echo 'WARN: Large diff ($LINES lines)' || echo 'OK: Diff size'

# 2. All tests pass
pytest tests/ -q --tb=no
[ $? -ne 0 ] && echo 'FAIL: Tests broken' && exit 1
echo 'OK: All tests pass'

# 3. Coverage check
pytest --cov=src/sensor_toolkit --cov-report=json -q
echo 'OK: Coverage report generated'

# 4. Lint check
LINT=$(ruff check src/ --output-format json 2>/dev/null | python3 -c 'import sys,json;
print(len(json.load(sys.stdin)))')
[ "$LINT" -gt 0 ] && echo "WARN: $LINT lint violations" || echo 'OK: Lint clean'

echo '=== Quality Gate Complete ==='
```

☆ **KEY POINT:** Integrate quality gates into your agent pipelines. After every agent execution step, run the gate script. If it fails, the pipeline stops and escalates to human review. This prevents agents from accumulating technical debt.

> ⚠ **WARNING:** Never trust agent-generated code blindly. Even with 100% test pass rate, an agent can introduce subtle logic errors that tests don't catch (this is exactly what mutation testing from Module 8 addresses). Quality gates are necessary but not sufficient — human review of non-trivial changes remains essential.

0:34 – 0:43	SECTION 5: MCP Server Integration

5. MCP Server Integration

5.1 What Is MCP?

Model Context Protocol (MCP) is a standard that lets Claude Code connect to external tools and data sources. Think of MCP servers as plugins that extend Claude Code's capabilities beyond reading and writing local files.

Without MCP	With MCP
Claude reads local files only	Claude reads from databases, APIs, cloud storage
Manual copy-paste from browser	Claude searches the web, reads documentation
Can't interact with issue trackers	Claude reads/writes Jira, GitHub Issues, Linear
Can't query databases directly	Claude runs SQL queries via database MCP server
Can't control browsers	Claude tests web UIs via Playwright MCP server

5.2 Adding MCP Servers

```
# Add an MCP server to your project:
claude mcp add github-server -- npx @anthropic-ai/github-mcp

# List all configured MCP servers:
claude mcp list

# MCP servers are stored in .mcp.json:
cat .mcp.json
# {
#   "mcpServers": {
#     "github-server": {
#       "command": "npx",
#       "args": ["@anthropic-ai/github-mcp"],
#       "env": { "GITHUB_TOKEN": "..." }
#     }
#   }
# }

# Scope options:
claude mcp add --scope user    # Available in all projects
claude mcp add --scope project # This project only (default)
```

5.3 MCP Servers for TI Workflows

MCP Server	What It Does	TI Use Case
GitHub	Read/write issues, PRs, repos	Automated PR creation and issue triage
Filesystem	Extended file operations	Bulk file processing across directories

PostgreSQL/SQLite	Run SQL queries	Query sensor data databases directly
Playwright	Browser automation	Test internal web dashboards
Slack	Read/write messages	Post CI results to team channels
Fetch/Web	HTTP requests	Query internal TI APIs for sensor specs

```
# Use MCP tools in subagent configuration:
# .claude/agents/data-analyst.md
---
name: data-analyst
description: Analyzes sensor data from production database.
tools: Read, Grep, Glob, mcp__postgres__query
model: sonnet
---
You are a data analyst with access to the TI sensor
production database. When asked about sensor data:
1. Query the database using mcp__postgres__query
2. Analyze results for anomalies
3. Generate a summary report
```

☆ **KEY POINT:** MCP servers turn Claude Code from a local development tool into a connected platform. With the right MCP servers, your subagents can read issues from GitHub, query production databases, test web UIs, and post results to Slack — all within the same agent workflow.

Track A (IT Engineers)	Track B (Process Engineers)
Set up GitHub MCP for automated PR workflows. Add Playwright for UI testing.	Set up PostgreSQL MCP for querying sensor data. Add Filesystem MCP for batch processing.
Create a subagent that uses GitHub MCP to read issue descriptions and generate implementation plans.	Create a subagent that uses database MCP to analyze sensor readings and generate quality reports.

0:43 – 0:45	WRAP-UP: Quick Reference & Transition to Lab 2

6. Quick Reference & Transition to Lab 2

MODULE 11 RECAP

✅ Observe first: agents must understand the codebase (structure, patterns, CLAUDE.md) before making changes.

✅ File localization: use Grep, Glob, @-references, and git log to find affected files before modifying anything.

✅ Plan-Execute-Review: plan mode (Shift+Tab) creates an approval gate. Plan → approve → execute → test → review → commit.

✅ Chain subagents through the cycle: planner (read-only) → implementer (write) → tester (verify) → reviewer (audit).

✅ Quality gates: diff size, test pass rate, coverage delta, lint violations, type check. Automate and gate on failure.

✅ MCP servers extend Claude Code: databases, APIs, issue trackers, browsers, Slack. Add via claude mcp add.

✅ MCP in subagents: include mcp__server__tool in the tools field for external data access.

QUICK REFERENCE CARD

Technique	Command / Pattern	Purpose
Observe repo	> Analyze this repository structure and patterns	Before any changes
Grep localization	Grep("SensorReading", "src/")	Find all references
Glob localization	Glob("tests/**/test_*.py")	Find test files
Impact analysis	> Find all files importing from validators.py	Before modifying a module
Plan mode	Shift+Tab → describe change	Create approval gate
Quality gate	pytest + ruff + git diff --stat	After every agent step
Add MCP server	claude mcp add name -- command	Extend capabilities
MCP in subagent	tools: Read, mcp__postgres__query	External data access
List MCP servers	claude mcp list	Check configuration

TRANSITION TO LAB 2

You now have all the pieces: subagents (Module 10), observation and localization, plan mode, quality gates, and MCP integration. In Lab 2 (Module 12), you'll combine everything into a hands-on mini assistant loop: summarize a repository, propose a change plan, implement it with an approval gate, run tests, and draft a PR — all using the techniques from Modules 10–11.